Reverse-engineering gene networks

Classroom R exercises using *in- silico* validation datasets

Roberto Pagliarini, University of Udine
Grete Francesca Privitera, University of Catania
Dora Tortarolo, University of Torino

GeneNet | glasso / huge | minet / C3NET | GENIE3

Learning path

Expression matrix -> preprocessing -> network inference -> graph object -> validation against the gold standard.

Class output

Each student produces edge lists, graphs and a short method comparison.

Available datasets

Expression datasets

Three TSV matrices: 100 samples × 100 genes.

- insilico_size100_1_knockdowns.tsv
- insilico_size100_2_multifactorial.tsv
- insilico_size100_3_multifactorial.tsv

Gold standards

- Three binary adjacency matrices: 100 genes × 100 genes.
- They can be used to validate inferred networks with precision, recall and F1.

Number of edges in each gold standard

Teaching note

The gold standard matrices are binary. The inference methods produce scores or edges, which must be thresholded before evaluation.

Lesson objectives

- Load expression and gold standard matrices from TSV files
- Check matrix orientation, dimensions, missing values and variability.
- Infer networks
- Convert score matrices into edge lists and igraph objects.
- Evaluate inferred networks against a binary gold standard.

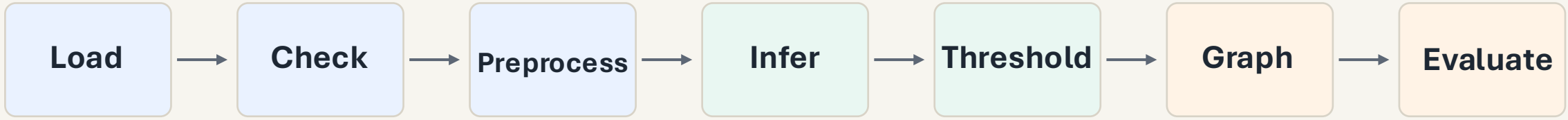
Final output

Ranked edge list networks + basic validation metrics.

Classroom format

Each exercise has a short task, starter code and interpretation questions.

Common workflow




```
# One dataset, many inference methods
# Keep the loading and evaluation
functions identical
# Change only the network inference
block
```

Class rule

Change one method at a time. This helps students see whether the result changes because of the method or because of preprocessing.

Exercise 0: environment setup

```
# Install CRAN packages
install.packages(c("igraph", "qgraph", "glasso",
"huge"))

# Install Bioconductor packages
if (!requireNamespace("BiocManager", quietly =
TRUE))
  install.packages("BiocManager")

BiocManager::install(c("GeneNet", "minet",
"GENIE3","BDgraph", "c3net"))
```

```
# Load packages for the
exercises
library(igraph)
library(qgraph)
library(GeneNet)
library(glasso)
library(huge)
library(minet)
library(GENIE3)
library(BDgraph)
```

Exercise 1: load a dataset and gold standard

```
# Set the project folder
base_dir <- ”Data/ValidationData"

expr_file <- file.path(base_dir, "DataSet",
  "insilico_size100_2_multifactorial.tsv")

gold_file <- file.path(base_dir, "GoldStandard",

"insilico_size100_multifactorial_2_goldstandard_matrix.tsv
")

# Read expression data: rows = samples, columns = genes
expr <- read.delim(expr_file, check.names = FALSE)

# Read gold standard: rows = genes, columns = genes
gold <- read.delim(gold_file, row.names = 1, check.names =
FALSE)
```

```
# Quick checks
dim(expr)
dim(gold)
head(colnames(expr))


# Convert to numeric matrices
X <- as.matrix(expr)
G <- as.matrix(gold)
storage.mode(X) <- "numeric"
storage.mode(G) <- "numeric"
```

Exercise 2: checks and minimal preprocessing

```
# Check missing values and basic ranges
sum(is.na(X))
summary(as.vector(X))

# Keep genes that vary across samples
gene_var <- apply(X, 2, var)
X <- X[, gene_var > 0]

# Standardize genes for covariance-based methods
X_scaled <- scale(X)

# Align the gold standard to the retained genes
common_genes <- intersect(colnames(X_scaled),
rownames(G))
X_scaled <- X_scaled[, common_genes]
G <- G[common_genes, common_genes]
```

Discussion point

These in silico data are already continuous. With real RNA-seq data, normalization and transformation must be done before this step.

Expected output

A numeric matrix named X_scaled, ready for network inference.

Exercise 3: correlation network

```
# Compute an absolute Pearson correlation matrix
S <- abs(cor(X_scaled, method = "pearson"))
diag(S) <- 0

# Keep only the strongest edges
threshold <- quantile(S[upper.tri(S)], 0.98)
A_cor <- (S >= threshold) * 1
diag(A_cor) <- 0

# Build an undirected graph
g_cor <- graph_from_adjacency_matrix(A_cor,
  mode = "undirected", diag = FALSE)

# Basic graph statistics
vcount(g_cor)
ecount(g_cor)
sort(degree(g_cor), decreasing = TRUE)[1:10]
```

Questions

What happens if you use the 95th, 98th or 99th percentile as the threshold? Which genes become hubs?

Why this baseline matters

It is simple, fast and useful as a reference before trying more complex methods.

Exercise 4: GeneNet partial correlation network

```
# Estimate shrinkage partial correlations
pcor_mat <- ggm.estimate.pcor(X_scaled)

# Test and rank candidate edges
genet_edges <- network.test.edges(
  pcor_mat,
  direct = FALSE,
  plot = FALSE
)

# Keep the best-ranked edges
top_edges <- genet_edges[order(genet_edges$qval), ][1:200, ]

# Build an edge list for igraph
el <- data.frame(
  from = top_edges$node1,
  to = top_edges$node2,
  weight = abs(top_edges$pcor)
)

g_genet <- graph_from_data_frame(el, directed = FALSE)
```

Interpretation

An edge represents conditional association after accounting for the other genes. It does not prove causality.

Exercise 5: glasso and huge

```
# glasso requires a covariance matrix
S_cov <- cov(X_scaled)

# Try one regularization value
rho <- 0.15
fit_glasso <- glasso(S_cov, rho = rho)

# Non-zero precision matrix entries define edges
P <- fit_glasso$wi
A_glasso <- (abs(P) > 1e-6) * 1
diag(A_glasso) <- 0

g_glasso <- graph_from_adjacency_matrix(A_glasso,
  mode = "undirected", diag = FALSE)
```

```
# huge can fit a path of sparse networks
fit_huge <- huge(X_scaled, method = "glasso")

# Select one model with the rotation
information criterion
sel <- huge.select(fit_huge, criterion =
"ric")
A_huge <- sel$refit

g_huge <- graph_from_adjacency_matrix(A_huge,
  mode = "undirected", diag = FALSE)
```

Mini-task

Increase rho. Does the network become denser or sparser?

Exercise 6: minet mutual information network

```
# Estimate mutual information
mi <- build.mim(X_scaled, estimator = "spearman")

# Try three common algorithms
net_aracne <- aracne(mi)
net_clr <- clr(mi)
net_mrnet <- mrnet(mi)

# Convert a score matrix to an adjacency matrix
S_mi <- as.matrix(net_clr)
diag(S_mi) <- 0
threshold <- quantile(S_mi[upper.tri(S_mi)], 0.98)
A_mi <- (S_mi >= threshold) * 1

g_minet <- graph_from_adjacency_matrix(A_mi,
  mode = "undirected", diag = FALSE)
```

Student task

Build three networks: ARACNE, CLR and MRNET. Compare edge counts and hub genes.

Discussion

Mutual information measures statistical dependence, but it does not give regulatory direction.

Threshold choice

Use the same percentile threshold as in the baseline to keep comparisons fair.

Exercise 7: C3NET conservative MI network

```
# Load packages
library(c3net)
library(igraph)

# Infer the network. C3NET expects genes in rows and samples in columns
A_c3 <- c3net(t(X_scaled))

# Remove self-connections
diag(A_c3) <- 0

# Build the graph
g_c3 <- graph_from_adjacency_matrix(
  A_c3,
  mode = "undirected",
  weighted = TRUE,
  diag = FALSE
)

# Inspect the network
vcount(g_c3)
ecount(g_c3)

# Identify the most connected genes
sort(degree(g_c3), decreasing = TRUE)[1:10]
```

Interpretation

C3NET is intentionally conservative. It usually returns fewer edges than standard MI thresholding.

Teaching angle

This is a good place to discuss false positives and false negatives.

Exercise 8: validate against the gold standard

```
# Evaluate undirected adjacency matrices
evaluate_network <- function(A_pred, A_true) {
  A_pred <- (A_pred > 0) * 1
  A_true <- (A_true > 0) * 1
  diag(A_pred) <- 0
  diag(A_true) <- 0

  idx <- upper.tri(A_true)
  TP <- sum(A_pred[idx] == 1 & A_true[idx] == 1)
  FP <- sum(A_pred[idx] == 1 & A_true[idx] == 0)
  FN <- sum(A_pred[idx] == 0 & A_true[idx] == 1)

  precision <- TP / (TP + FP)
  recall <- TP / (TP + FN)
  f1 <- 2 * precision * recall / (precision + recall)

  data.frame(TP, FP, FN, precision, recall, f1)
}
```

```
# Example: evaluate three networks
G_bin <- (G > 0) * 1

evaluate_network(A_cor, G_bin)
evaluate_network(A_glasso, G_bin)
evaluate_network(A_mi, G_bin)
```

Class question

A denser network often increases recall. What happens to precision?